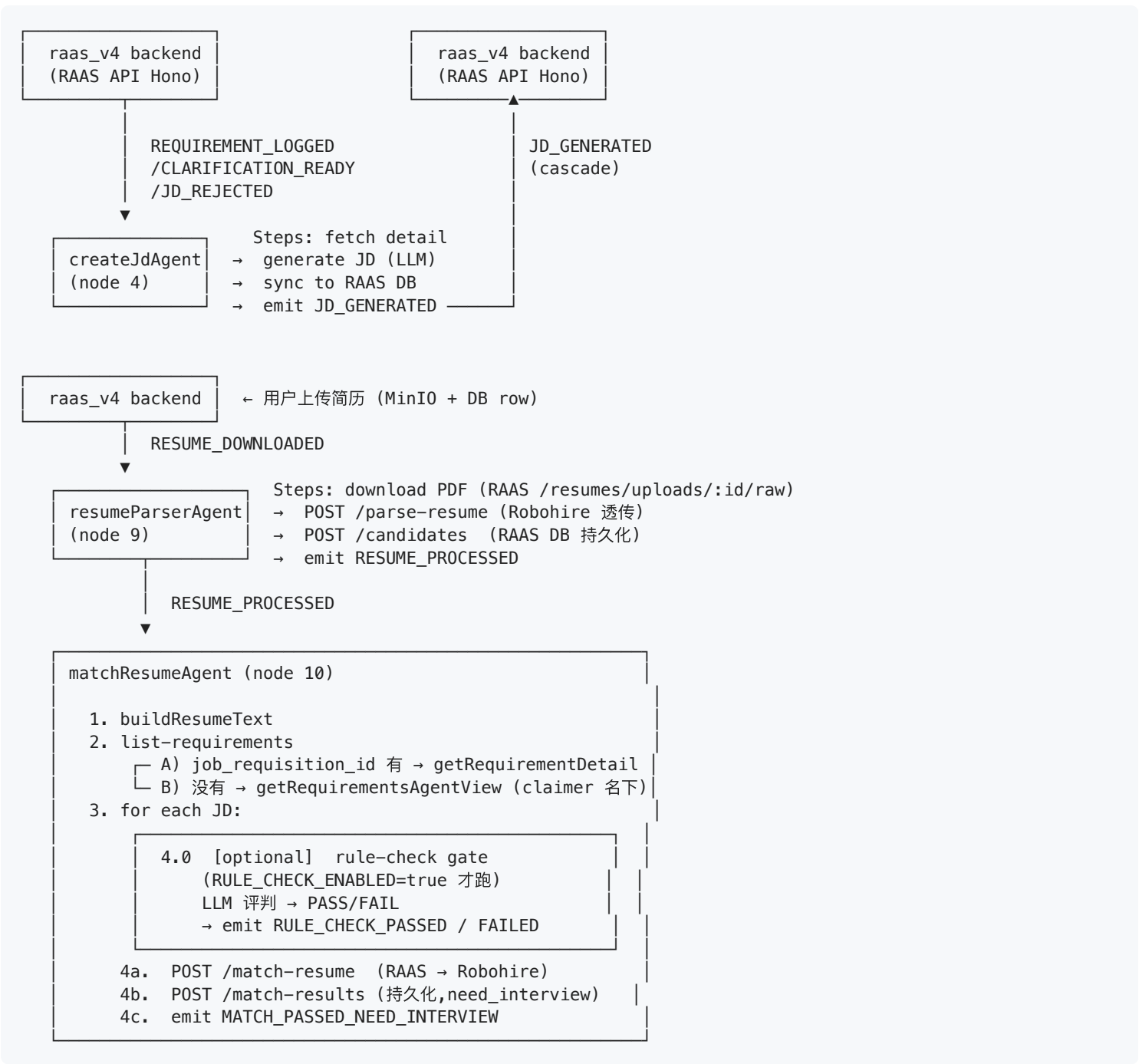


AO Workflow 端到端事件链 + Agent 执行说明

AO Workflow 端到端事件链 + Agent 执行说明

范围:Agentic Operator(AO main,端口 3002)的 Inngest 事件驱动 workflow 状态:当前现状 + 未来计划(Rule check 启用 + Neo4j 实例存储 + 叶洋 v4 prompt) 最后更新:2026-05-11(p4 合并后,resume-parser-agent 子项目已并入主仓)

0. TL;DR — 当前端到端事件链



3 个 agent, 8 种事件, 4 个外部 endpoint (RAAS API + LLM + Inngest + 未来的 Ontology API)。

1. 事件总览

1.1 当前在用的事件

事件名	方向	触发方	消费方	payload 核心字段
REQUIREMENT_LOGGED	RAAS → AO	raas_v4 backend	createJdAgent	entity_id , payload.raw_input_data (28 字段), trace
CLARIFICATION_READY	RAAS → AO	raas_v4 backend(澄清闭 环后)	createJdAgent	同上 shape
JD_REJECTED	RAAS → AO	raas_v4 backend(JD 审核 拒绝)	createJdAgent	同上 shape
JD_GENERATED	AO → RAAS	createJdAgent	raas_v4 backend(回写 JobPosting)	payload.{posting_title, posting_description, must_have_skills, ...}
RESUME_DOWNLOADED	RAAS → AO	raas_v4 backend(用户上 传)	resumeParserAgent	upload_id, bucket, objectKey, employeeId, job_requisition_id (optional)
RESUME_PROCESSED	AO → AO + RAAS	resumeParserAgent	matchResumeAgent + raas_v4	upload_id, candidate_id, resume_id, parsed.data, job_requisition_id
MATCH_PASSED_NEED_INTERVIEW	AO → RAAS	matchResumeAgent (per JD)	raas_v4 backend / UI	upload_id, job_requisition_id, data: {matchScore,...}
RULE_CHECK_PASSED	AO → AO	matchResumeAgent (gate enabled)	审计 / 未来 Neo4j	upload_id, job_requisition_id, audit:{rules_evaluated, llm_decision, llm_model, ...}
RULE_CHECK_FAILED	AO → AO	matchResumeAgent (gate enabled)	审计 / 未来 Neo4j	同上 + failure_reasons[], hit_rules[]

1.2 已声明、暂未发的事件

事件名	说明
MATCH_PASSED_NO_INTERVIEW	不需要内部面试的客户场景。当前 matchResumeAgent 没分支,默认都走 MATCH_PASSED_NEED_INTERVIEW 。
MATCH_FAILED	全部 JD 没命中的标记。当前用 summaries 字段汇报,没单独 emit。

2. 当前 3 个 Workflow Agents

2.1 resumeParserAgent — Workflow node 9

代码: `server/inngest/agents/resume-parser-agent.ts`

订阅: `RESUME_DOWNLOADED`

职责:从 MinIO 拉 PDF,用 Robohire `/parse-resume` 解析,持久化到 RAAS DB,emit 事件给下游 matcher

Steps:

#	step name	干什么	外部 IO
1	<code>unwrap</code> 解析 envelope	兼容两种 shape: <code>{entity_id, payload, trace}</code> 或 flat <code>{upload_id, bucket, ...}</code>	—
2	提取 anchor	<code>upload_id / bucket / object_key / employee_id / job_requisition_id</code> (可选)	—
3	<code>download-and-parse-\${upload_id}</code>	A) GET <code>/api/v1/resumes/uploads/:id/raw</code> 拿 PDF 字节 B) POST <code>/api/v1/parse-resume</code> (multipart) C) MD5 算 etag 兜底	RAAS API
4	<code>save-candidate</code>	POST <code>/api/v1/candidates</code> (parsed 数据 + transport context)	RAAS API
5	<code>emit-resume-processed</code>	sendEvent <code>RESUME_PROCESSED</code>	Inngest

关键边界: - `retries: 0` — RAAS API 失败不自动重试(避免重复扣 Robohire 配额、避免 dup write) - 4xx 抛 `NonRetriableError, 5xx / 429 / 网络错让 Inngest step 重试一次` - 兼容 legacy 路径:事件里如果已经带 `parsed.data` (老内部链路),跳过 step 3 - **Dual-track emit**:RAAS 那边 `saveCandidate` 后 partner 也会自动 emit `RESUME_PROCESSED` 。我们这边也 emit 一份做兜底。等 partner 全路径稳定后可以去掉。

2.2 createJdAgent — Workflow node 4

代码: `server/inngest/agents/create-jd-agent.ts`

订阅: `REQUIREMENT_LOGGED / CLARIFICATION_READY / JD_REJECTED` (三个事件 same shape,统一处理)

职责:从 RAAS 拉客户原始需求详情 → Robohire `/generate-jd` 生成 JD → 回写 RAAS DB → emit `JD_GENERATED` 给下游

Steps:

#	step name	干什么	外部 IO
1	提取 job_requisition_id from envelope	unwrap RAAS canonical envelope {payload:{raw_input_data}} 或 flat	—
2	fetch-requirement-\${id}	GET /api/v1/requirements/:id → 拿 RaasRequirement + RaasRequirementSpecification	RAAS API
3	generate-\${id}	POST /api/v1/generate-jd (透传 Robohire,prompt 由 raas-api-client 内部构造)	RAAS API (Robohire)
4	sync-jd-\${id}	POST /api/v1/jd/sync-generated (持久化 partner-canonical shape:写 JobPosting + 回填 JobRequisition + 推进 spec.status → pending_publish)	RAAS API
5	emit-jd-generated-\${id}	sendEvent JD_GENERATED (payload spread Robohire camelCase + RAAS snake_case enhancement)	Inngest

关键边界: - retries: 1 —— 一次重试,5xx 走 Inngest 标准 retry,4xx 抛 NonRetriableError - generator_version: 'workflow-a@2026-05-08' —— 标记当前生成器代际,以后 schema 改了能通过版本号区分 - JD_GENERATED.payload 形态跟 RAAS sync-generated body 1:1 对齐(doc v5 §4.6),raas 端不需要再做转换

2.3 matchResumeAgent — Workflow node 10

代码: `server/inngest/agents/match-resume-agent.ts`

订阅: RESUME_PROCESSED

职责:把候选人简历跟招聘人员名下的所有“在招”客户原始需求做匹配 → emit MATCH_PASSED_NEED_INTERVIEW (per JD)。可选先跑一道 LLM 预筛 gate。

Steps:

#	step name	干什么	外部 IO
1	unwrap event + 提取 anchor	upload_id , candidate_id , employee_id (claimer), job_requisition_id (可选)	—
2	build-resume-text	parsed.data → stringify 给 RAAS 用的 resume text	—
3	list-requirements	路径 A(jr_id 有值):GET /api/v1/requirements/:id 单 JD 精准匹配 路径 B(jr_id 空):GET /api/v1/requirements/agent-view? claimer_employee_id=... 拉 claimer 名下全 recruiting JD,客户端二层过滤(isRecruitingStatus + hasMatchableContent)	RAAS API
4	for each JD: ↓		
4.0	rule-check-\${jr_id} (可选)	RULE_CHECK_ENABLED=true 才跑。 buildRuleCheckInput → runRuleCheck (prompt 来源由 RULE_CHECK_PROMPT_SOURCE 选 poc / yeyang)→ LLM 评判 → binary PASS/FAIL	LLM 网关(AI_BASE_URL 或 OpenAI)
4.0a	(FAIL 路径) emit + skip	sendEvent RULE_CHECK_FAILED , continue 跳过这条 JD,不调 Robohire	Inngest
4.0b	(PASS 路径) emit + 继续	sendEvent RULE_CHECK_PASSED , 进入 4a	Inngest
4a	match-\${jr_id}	POST /api/v1/match-resume (透传 Robohire,resume_text + jd_text)	RAAS API (Robohire)
4b	save-match-\${jr_id}	POST /api/v1/match-results (持久化,source="need_interview",spread Robohire data 整段 + raas anchor)	RAAS API
4c	emit-match-\${jr_id}	sendEvent MATCH_PASSED_NEED_INTERVIEW	Inngest

关键边界: - retries: 2 - gate 默认关(RULE_CHECK_ENABLED env 不设 / 非 "true"):整个 4.0 block bypass,行为 = 接入 gate 前 - per-invocation 读 env: isRuleCheckEnabled() 是函数,每次调用都读 process.env ,Inngest cloud toggle 后下一次 invocation 立刻生效(不需要 redeploy) - 路径 A 的单 JD 来自 raas 前端"上传简历"弹框的"关联岗位(可选)"下拉。有值 = 用户精准指定;空 = fallback 到 claimer 名下扫描 - 路径 A 不再做 isRecruitingStatus 兜底过滤(用户明确选了即使岗位关闭也匹配),但仍要 hasMatchableContent - 4a 的 Robohire 4xx(非 429)→ 跳过这条 JD,不影响其他 JD 继续匹配 - 4b 用 ...matchResult.data spread 整段透传(20+ 字段),不 cherry-pick(之前漏过 14+ 字段导致 match_analysis 列空着)

3. 外部依赖

3.1 RAAS API Server

Endpoint	谁调	干什么
GET /api/v1/resumes/uploads/:id/raw	resumeParserAgent	下载 PDF 字节(替代直连 MinIO)
POST /api/v1/parse-resume	resumeParserAgent	透传 Robohire /parse-resume ,multipart 上传
POST /api/v1/candidates	resumeParserAgent	持久化候选人(返回 candidate_id , resume_id)
GET /api/v1/requirements/:id	createJdAgent / matchResumeAgent	拿单条客户原始需求详情 + spec
GET /api/v1/requirements/agent-view?claimer_employee_id=	matchResumeAgent	拉招聘人员名下所有招聘中 JD
POST /api/v1/generate-jd	createJdAgent	透传 Robohire /generate-jd 生成 JD
POST /api/v1/jd/sync-generated	createJdAgent	持久化 JD partner-canonical shape
POST /api/v1/match-resume	matchResumeAgent	透传 Robohire /match-resume 打分
POST /api/v1/match-results	matchResumeAgent	持久化匹配结果 (source= need_interview)

所有调用走 `lib/raas-api-client.ts` ,统一 `RaasApiError` 异常分类(4xx isClientError → NonRetriable;5xx/429/网络 → retry)。

3.2 LLM 网关(仅 rule-check gate 启用时)

`lib/rule-check/llm.ts` : - 优先 `AI_BASE_URL + AI_API_KEY` (默认指向 New-API 网关,model `google/gemini-3-flash-preview`) - Fallback `OPENAI_API_KEY` (`baseUrl=https://api.openai.com/v1` , model `gpt-4o-mini`) - 两个都没配 → 抛错

3.3 Inngest dev server

- Local docker (`docker-compose.inngest.yml`):一个容器,启动时 `-u http://host.docker.internal:3002/api/inngest` 主动同步本地 SDK
- Fallback local CLI(Docker 不可用): `npm run inngest:dev` 走 devDep 里的 `inngest-cli`
- 注册: `scripts/register-with-inngest.ts` POST `/fn/register` 推 endpoint(autodiscovery 扫不到 3002 时手动跑)

3.4 Ontology API(未来,planned)

`:3500/api/v1/ontology/*` — 主仓的 ontology metadata + 未来的 instance data。详见 `docs/neo4j-instance-storage-plan.md`。

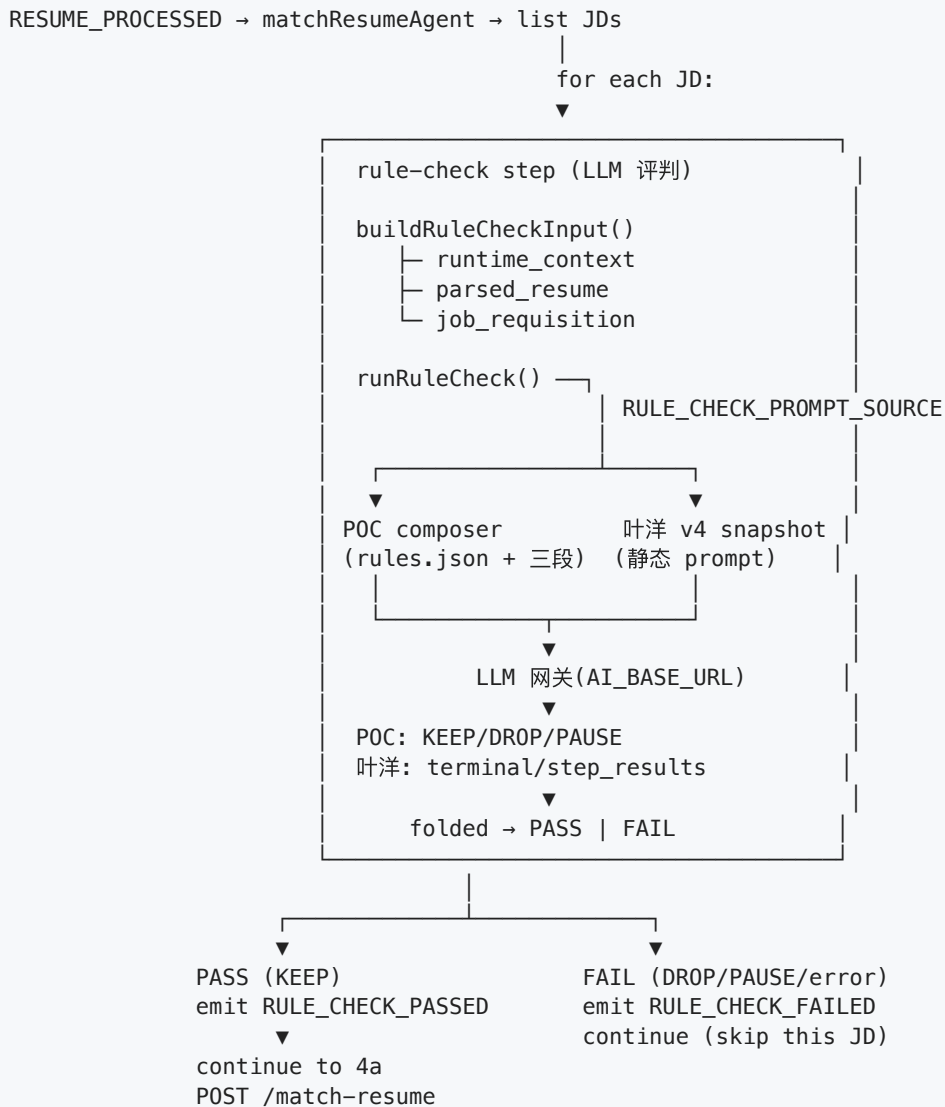
4. 未来的端到端事件链(planned)

3 个增量,各自可独立启用,都默认关:

4.1 增量 A:Rule check gate 启用(代码已落地,等切开关)

触发条件:Inngest cloud 设 `RULE_CHECK_ENABLED=true`

事件链改动: `matchResumeAgent` 的 for-loop 在每条 JD 上新加 step 4.0(LLM 预筛 gate)



新事件流向: - `RULE_CHECK_PASSED`:per-JD audit signal,目前没人订阅 - `RULE_CHECK_FAILED`:per-JD reject signal,带 `failure_reasons[]` + `hit_rules[]`

收益: - 在调 Robohire 之前过滤掉显然违反业务规则的(腾讯历史从业、华为荣耀竞业冷冻、CSI 黑名单、外籍通道限制等 51 条) - 不消耗 Robohire 配额

风险: - LLM 误杀率 — 需要在真实数据上跑通至少 N 个场景验证准确率 $\geq 5/6$ 才开 - LLM 网关 outage → FAIL-safe 全 reject,需要 alerting

4.2 增量 B:Neo4j 实例数据存储(代码未落地)

触发条件: `ENABLE_NE04J_INSTANCE_WRITE=true` (待加)+ Ontology API token 配好

事件链改动:在 `RULE_CHECK_PASSED` / `RULE_CHECK_FAILED` 之后,把 audit 写入 Neo4j(走 Ontology API,不直连 driver)

matchResumeAgent (gate enabled)

rule-check step done

emit RULE_CHECK_PASSED / FAILED

写 audit 到 Neo4j 通过 Ontology API
POST :3500/api/v1/ontology/instances/
RuleCheckAudit
+ POST ../RuleCheckFlag (per hit)
+ (条件) POST ../CandidateBlacklist

Add new step:

write-rule-check-audit- $\{jr_id\}$

- audit, links to:RuleSet
- 命中规则 + reasoning + evidence
- 跨次累计被拒后自动加黑名单

(rest of for-loop:matchResume → save → emit)

新增 **DataObject schemas**(在 Ontology API 注册): - RuleCheckAudit — 每次 rule-check 一条,记录决策上下文 + LLM 元数据 - RuleCheckFlag — 命中的每条规则一条,链回 ontology Rule 节点 - CandidateBlacklist — 派生规则(候选人被同一规则拒 N 次,加客户级黑名单)

关键设计: - 不复制 **RAAS DB** 已有的可变业务字段(候选人电话、当前简历正文、岗位最新分数) - 要 **snapshot** 决策上下文: client_name / business_group_code / ruleset_version / rule_name_snapshot 等(point-in-time 语义 + 查询性能 + 跨系统鲁棒性) - ID 引用(candidate_id , job_requisition_id)指向 RAAS DB,不复制

详见 <docs/neo4j-instance-storage-plan.md>。

4.3 增量 C:叶洋 v4 prompt 完全启用(已搭建,等切开关)

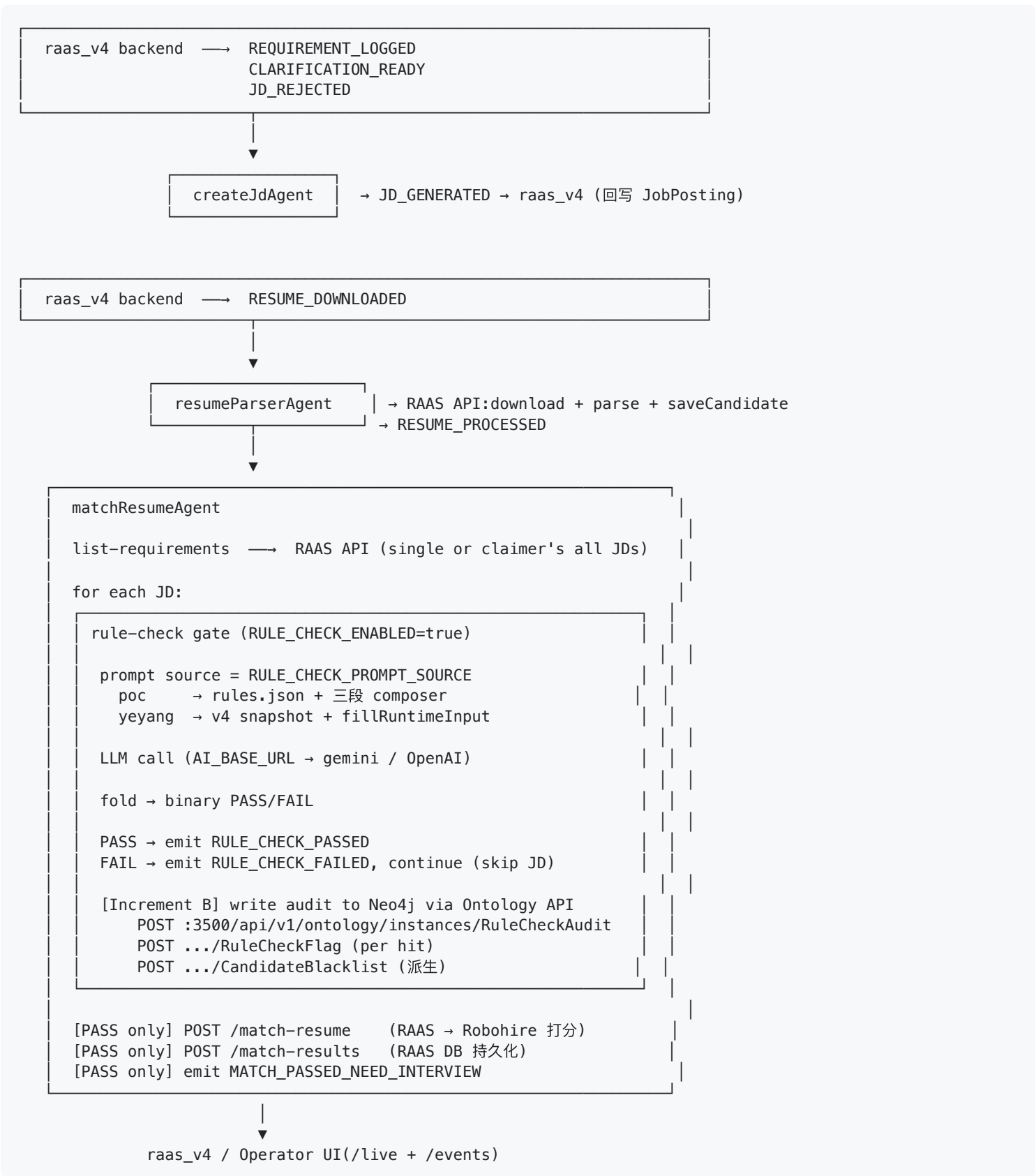
触发条件: RULE_CHECK_ENABLED=true + RULE_CHECK_PROMPT_SOURCE=yeyang

事件链改动:rule-check step 内部的 prompt 来源切换。对外事件链不变(还是 RULE_CHECK_PASSED/FAILED),但 prompt 内容和 LLM 输出 schema 变了。

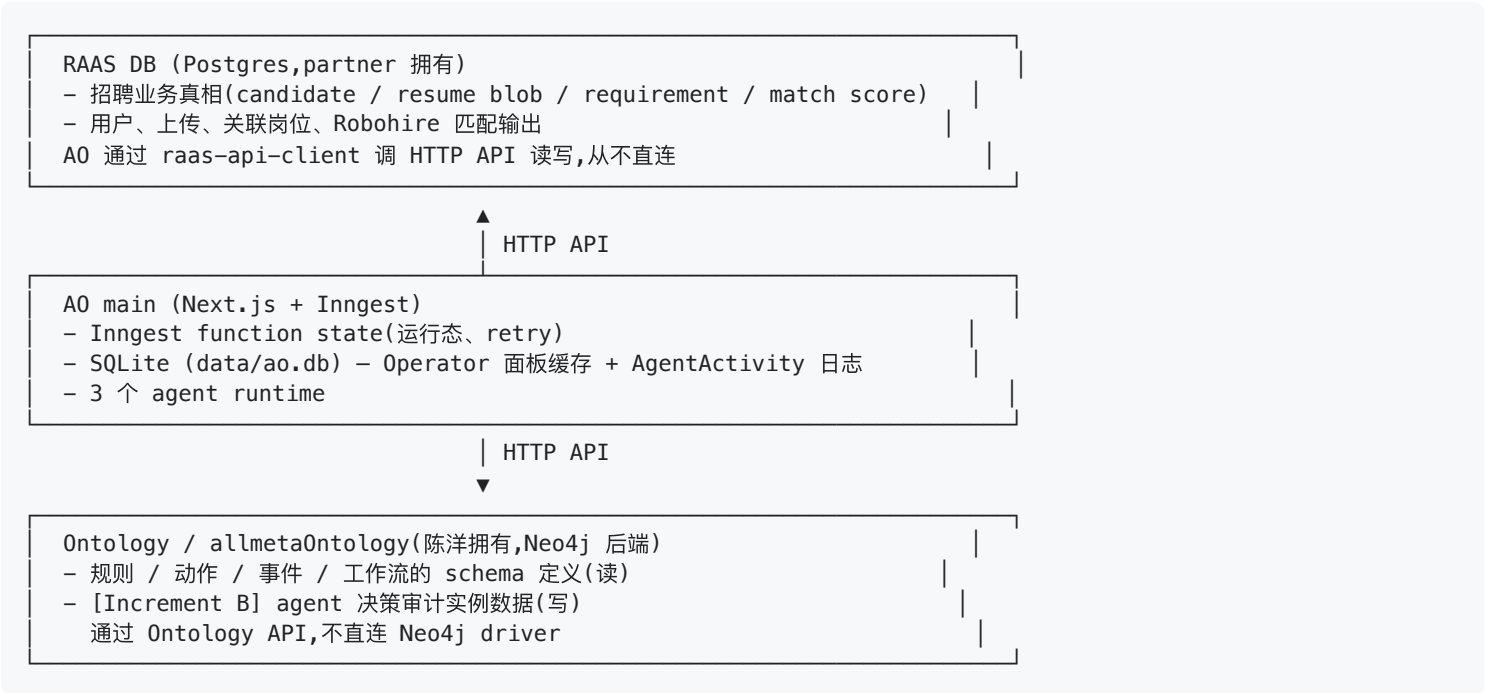
对比	POC 路径(默认)	叶洋 v4 路径
Prompt 来源	<code>lib/rule-check/prompt.ts</code> 渲染 <code>rules.json</code>	<code>generated/v4/match-resume.action-object.ts</code> 静态 snapshot + <code>fillRuntimeInput()</code>
过滤维度	code-side filter: (client × business_group) 由 <code>ontology.ts</code> 做	主仓 snapshot 生成时 baked-in,运行时直接用
Severity 推断	文本关键词推断(<code>inferSeverity</code>)	主仓代际侧已处理
Prompt 结构	1. 角色 / 2. Inputs(5 块) / 3. Rules(3 类) / 4. 决策 / 5. Output schema / 6. 自检	1. 角色 / 2. 约束 / 3. 任务 / 4. 运行时输入(3 placeholders) / 5. 输出 JSON / 6. Steps 1-4 / 7. 自检
LLM 输出 schema	<code>{ overall_decision: KEEP\ DROP\ PAUSE, drop_reasons, pause_reasons, rule_flags, ... }</code>	<code>{ match_results, overall_status, terminal, step_results: { step_1..4: { status, fired_rule_ids, blocking_rule_ids, notifications } } }</code>
Binary 折叠规则	<code>KEEP → PASS;DROP/PAUSE → FAIL</code>	<code>terminal=true \ \ </code> <code>overall_status='全部不匹配' \ \ any</code> <code>step.status='blocked' \ \ </code> <code>pending_human → FAIL;其他 → PASS</code>
状态	✓ POC 验过 6 场景(4/6 准确率)	△ 集成完,未在真实数据上交叉验证

详见 `docs/rule-check-user-guide.md` 。

5. 总图:未来全启用状态(增量 A + B + C 都开)



6. 数据落到哪里(分层职责)



铁律:Neo4j 不复制 RAAS DB 拥有的可变业务字段。只放 ID 引用 + agent 决策 + snapshot 的决策上下文(client_name 、 business_group_code 、 rule_name@version 等)。

7. 操作 checklist — 各增量启用顺序

不要全开。建议:

1. 当前(默认):所有 gate / 写 Neo4j 关闭,RAAS partner 走原 RESUME_PROCESSED → matchResume → MATCH_PASSED_NEED_INTERVIEW 链路
2. 第 1 步(建议):本地启用 RULE_CHECK_ENABLED=true (RULE_CHECK_PROMPT_SOURCE=poc),发测试事件验证 gate 行为合理
3. 第 2 步:在真实数据上跑一周,看 RULE_CHECK_FAILED 命中率 + 误杀率,准确率 ≥ 5/6 才考虑生产开
4. 第 3 步(可选):切到 RULE_CHECK_PROMPT_SOURCE=yeyang 跑同样测试,对比两条路径差异
5. 第 4 步:Inngest cloud 设 RULE_CHECK_ENABLED=true 灰度启用
6. 第 5 步:实施增量 B(Neo4j 实例存储),先把 Ontology API token / domain / schema 落地
7. 第 6 步:全启用 — RULE_CHECK_ENABLED=true + Neo4j 实例写 + 监控覆盖全

回滚预案(对应任意一步): - 任意时刻 RULE_CHECK_ENABLED=false → gate 完全 bypass,行为 = 第 0 步 - Neo4j 写失败 → 走 try/catch,不影响主流程(只丢审计,不丢业务) - LLM 网关 outage → rule-check FAIL-safe(全 reject),需要 alert 但不引起雪崩

8. 相关文档

文档	内容
docs/rule-check-user-guide.md	Rule check gate 详细用户指南(POC 路径 / 叶洋路径)
docs/neo4j-instance-storage-plan.md	Neo4j 实例数据存储方案(Ontology API 路径,3 个新 schema)
docs/yeyang-prompt-adapter-onboarding.md	给叶洋的 adapter onboarding(v4 generatePrompt 设计)
docs/workflow-agents-inngest-spec.md	早期 Inngest agent 设计 spec(部分内容已被本文档替代)

9. 改动这里要 keep in mind 什么

- 加新 step 时:Inngest `step.run` 是 idempotent 的,返回值会被序列化 **cache**(retry 时不重跑)。Buffer / 自定义 class 不能直接 return,要 primitive 化
- emit 新事件时:类型加在 `server/inngest/client.ts` 的 `Events` 注释里(虽然 Inngest 4 不再用 EventSchemas 强校验,但 TS 类型给开发期 narrowing)
- 改 agent retry policy 要小心:matchResumeAgent 的 `retries: 2` 是 LLM 调用 + RAAS API 的 5xx 兜底,改成 0 会让短时网络抖动失败
- gate 默认 false 不变 — 任何“启用”动作走 Inngest cloud env,不要在代码里 **hardcode true**
- LLM 调用结果落 Neo4j 时,**Neo4j** 写失败不影响主流程(只丢审计,主流程继续。这点 Increment B 实现时要注意 try/catch)